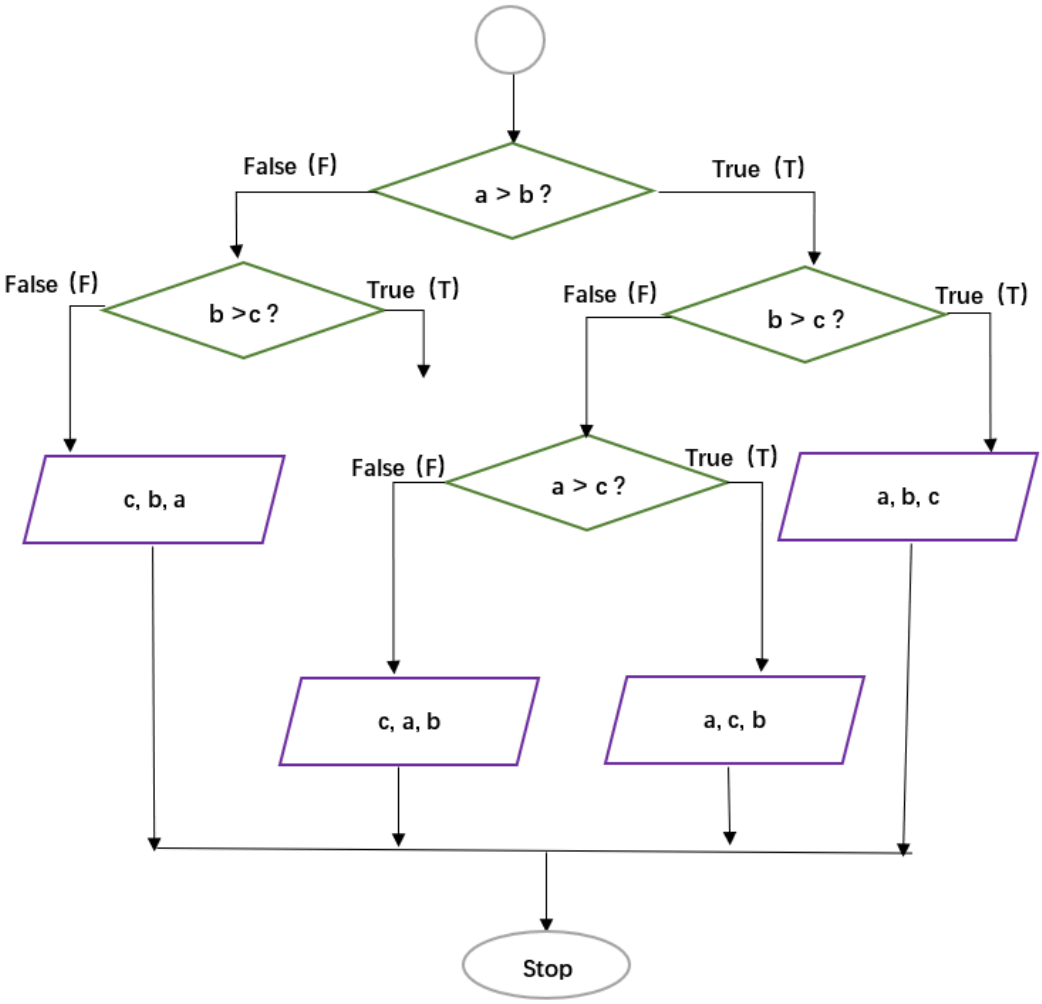
1. Flowchart

Write a function with arguments Print_values a, b and c to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random a, b, c and values.



In [2]:
```
def Print_values(a, b, c):
    if a > b:
        if b > c:
            print(f"{a},{b},{c}")          # a > b > c
        else:
            if a > c:
                print(f"{a},{c},{b}")      # a > c > b
            else:
                print(f"{c},{a},{b}")      # c > a > b
    else:
        if b > c:
           pass
        else:
            print(f"{c},{b},{a}")          # c > b > a
```

In [3]:
```
a = float(input("请输入a: "))
b = float(input("请输入b: "))
c = float(input("请输入c: "))
```

```
Print_values(a, b, c)
```

```
10.0,9.0,6.0
```

2. Matrix multiplication

2.1 Make two matrices M1(5 rows and 10 columns) and M2(10 rows and 5 columns); both are filled with random integers from 0 and 50.

In [4]:
```
import random
import numpy as np
```

In [5]:
```
M1 = np.random.randint(0, 51, size=(5, 10))    # 5 行 10 列
M2 = np.random.randint(0, 51, size=(10, 5))    # 10 行 5 列

print(M1)
print(M2)
```

```
[[11  8  9 43 21  9 44 18 17 49]
 [50 49  1 24 39 37 28 16 26  3]
 [19 47 44  8  2 47 25 31 10  6]
 [31  3 25 30 43 50 31 30 46  7]
 [41 31 22 49 28 30 26  3  2 21]]
[[25 27 29  5 12]
 [40 45 35 16 23]
 [27  4  0 28 15]
 [49 33 10 18 37]
 [30 35 10 30 10]
 [38 19 20 39 16]
 [13 11 47  5  3]
 [45  4 19 44 40]
 [46  8 33 47 14]
 [46 34 24  7 13]]
```

2.2 Write a function to do matrix multiplication, i.e.Matrix_multip, M1 * M2. Here you are ONLY allowed to use for loop, operator *, and operator +.

In [6]:
```
def Matrix_multip(M1, M2):
    rows = len(M1)          # 最终行数
    cols = len(M2[0])       # 最终列数
    n = len(M2)             # 中间值

    result = []
    for i in range(rows):
        row = []
        for j in range(cols):
            total = 0
            for k in range(n):
                total = total + int(M1[i][k]) * int(M2[k][j])
            row.append(total)
        result.append(row)

    return result
```

In [7]:
```
print("M1 * M2 = ")
for row in Matrix_multip(M1, M2):
    print(row)
```

```
M1 * M2 =
[8335, 5376, 5566, 4344, 4123]
[9407, 7101, 7085, 6194, 4739]
[8237, 4714, 5474, 6117, 4570]
[10421, 5584, 6447, 8369, 5184]
[8771, 6785, 5493, 4712, 4607]
```

3. Pascal triangle

One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function with an argument to print the k Pascal_triangle kth line of the Pascal triangle. Report Pascal_triangle(100) and Pascal_triangle(200) .

In [8]:
```python
def Pascal_triangle(t):
    row = [1]                # 第 0 行 [1]

    for i in range(1, t + 1):
        new_row = [1]        # 每行开头是 1
        for j in range(1, i):
            new_row.append(row[j - 1] + row[j])     # 中间的数 = 上一行相邻两个数
        new_row.append(1)    # 每行结尾是 1
        row = new_row

    return row
```

In [9]:
```python
print("Pascal_triangle(100): ")
print(Pascal_triangle(100))

print("\nPascal_triangle(200): ")
print(Pascal_triangle(200))
```

```
Pascal_triangle(100):
[1, 100, 4950, 161700, 3921225, 75287520, 1192052400, 16007560800, 186087894300,
1902231808400, 17310309456440, 141629804643600, 1050421051106700, 711054249979920
0, 44186942677323600, 253338471349988640, 1345860629046814650, 665013487293720180
0, 3066451080298820830 0, 13234157293921226740 0, 53598337040380968 2041841411
062132125600, 7332066885177656269200, 2486527030625466039120 0, 7977607556590036 87
55100, 2425192697203371210155 04, 6995748165009724644678 00, 1917353200780443050763
600, 4998813702034726525205100, 1241084781194828654533 6800, 2937233982161094482 39
63760, 6632463830686342379604 7200, 1430125013491742575602 26775, 2946924270225408 9
4366527900, 5807174297208894094869 81450, 1095067153187962886461165020, 1977204582
144932989443770175, 3420029547493938143902737600, 5670048986634686922786117600, 9
013924030034630492634340800, 1374623414580281150126 7369720, 2011644021336996805 06
35175200, 2825880887116257416636 8460400, 3811653289598672794533 4202400, 493782357
97073715747364762200, 6144847121413617959672059 2960, 7347099819081499734390505680
0, 8441348728306403950150 7937600, 9320655887504987694958 1681100, 9891308288780803
2681188722800, 1008913445455641933348 12497256, 9891308288780803268118 8722800, 932
065588750498769495816 81100, 8441348728306403950150 7937600, 7347099819081499734390
5056800, 6144847121413617959672059 2960, 4937823579707371574736 4762200, 3811653289
5986727945334202400, 2825880887116257416636 8460400, 2011644021336996805 06351752 0
0, 1374623414580281150126 7369720, 9013924030034630492634340800, 5670048986634686 9
22786117600, 3420029547493938143902737600, 1977204582144932989443770175, 10950671
5318796288646116 5020, 5807174297208894094869 81450, 2946924270225408943665279 00, 1
430125013491742575602 26775, 6632463830686342379604 7200, 2937233982161094482396376
0, 1241084781194828654533 6800, 4998813702034726525205100, 1917353200780443050763 6
00, 6995748165009724644678 00, 2425192697203371210155 04, 7977607556590036875510 0,
2486527030625466039120 0, 7332066885177656269200, 2041841411062132125600, 53598337
0403809682970, 132341572939212267400, 30664510802988208300, 6650134872937201800,
1345860629046814650, 253338471349988640, 44186942677323600, 7110542499799200, 105
0421051106700, 141629804643600, 17310309456440, 1902231808400, 186087894300, 1600
7560800, 1192052400, 75287520, 3921225, 161700, 4950, 100, 1]

Pascal_triangle(200):
[1, 200, 19900, 1313400, 64684950, 2535650040, 82408626300, 2283896214600, 550989
96177225, 1175445251780800, 22451004309013280, 387790074428411200, 61076936722474
76400, 88326646952501966400, 1179791641436990551200, 1462941635381868283488 0, 169
152626591028520278300, 1830828428985249866541600, 1861342236135004030983960 0, 178
296993145563544020568800, 1613587787967350073386147640, 1383075246829157205759555
1200, 1125320314465541544686183484 00, 8709000694559408476266985224 00, 64228880122
37563751246901602700, 4521713160615244880877818728 3008, 3043460781183337900590839
52866400, 1961341392318151091491874362916800, 1211828788825143352957479517087880
0, 7187398333721539886506430239279 8400, 4096817050221277735308665236389508 80, 224
664160818586198387894545221360 1600, 1186507599323158360236068066950308 3450, 60404
023238269880157472556135652061200, 2966903494350314701852328492545263006 00, 14071
599430347206871642472278928961685 60, 6449483072242469816169466461175774105900, 28
586897941831487833832229719806133874800, 1226227464346982241293329853770631531998
00, 5093560236518233925372293238739546363684 00, 2050157995198589154962348028592666
7411382810, 8000616566628640604731114257922604532225600, 3028804843080842514648 20
754049929717157711200, 1112909686527379342591666956741592863004272 00, 397106410874
542174515662982291886544299251600, 1376635557698412871654298338611873353570738888
0, 4638663292244652067530787880105225430510098400, 1519902440437609400850513475 60
894620489054288 00, 4844689028894879965211011703503516028088605430 0, 1502842311004
126030024640365168437625039730664 00, 4538583779232460610674413902808681627619986 6
0528, 1334877582127194297257180559649612243417643119200, 3824937687249075967140 76
737284215815902363124540 0, 1068095807005402345541195417321961146293391366640 0, 29
075941412924841628621430804875608982431209425200, 7718340811430957959597688904566
9798389726483201440, 1998498960102658757395830162789664422591132154323 00, 5048839
4781540852818420972533633627518091759688160 0, 1244800078234541716040379150398208 4
02601227868173600, 2995959510327202096232776938246535477447023004756800, 70405048
4926892492614702580487935837200050406117848 0, 1615853571963359819115710840464115 0
361968369976475200, 3622639459724306691243287206846967581150973268919440 0, 793530
```

548320562418081862911976002422537832239858544000, 169865132999870392620648779594863018574504713844719575, 79353054832056241808186291197600242253783223985854400, 362263945972430669124328720684696758115097326891944000, 161585357196335981911571084046411503619683699764752000, 7040504849268924926147025804879358372000504061178480, 2995959510327202096232776938246535477447023004756800, 124480007823454171604037915039820840260122786817360048839478154085281842097253363362751809175968816001998498960102658757395830162789664422591132154323007718340811430957959597688904566979838972641

```
83201440, 290759414129248416286214308048756089824312094252000, 1068095807005402345
541195417321961146293391366640 0, 382493768724907596714076737284215815902363124540
0, 133487758212719429725718055964961224341764311920 0, 453858377923246061067441390
280868162761998660528, 150284231100412603002464036516843762503973066400, 48446890
288948799652110117035035160280886054300, 151990244043760940085051347560894620489 0
5428800, 463866329224465206753078788010522543051009840 0, 137663555769841287165429
833861187335357073888 0, 397106410874542174515662982291886544299251600, 1112909686
527379342591666956741592863004272 00, 302880484308084251464820754049927171577112 0
0, 800061656662864060473111425792260453222560 0, 205015799519858915496234802859266
7411382810, 509356023651823392537229323873954636368400, 122622746434698224129332 9
85377063153199800, 285868979418314878338322297198061338748 00, 64494830722424698 16
169466461175774105900, 140715994303472068716424722789289616 8560, 2966903494350314
701852328492545263006 00, 604040232382698801574725561356520612 00, 1186507599323158
360236068066950308345 0, 224664160818586198387894545221360160 0, 409681705022127773
530866523638950880, 718739833372153988650643023927984 00, 121182878882514335295747
95170878800, 196134139231815109149187436291680 0, 304346078118333790059083952866400,
452171316061524488087781872830 08, 642288801223756375124690160270 0, 87090006945
5940847626698522400, 112532031446554154468618348400, 138307524682915720575955512 0
0, 161358778796735007338614764 0, 178296993145563544020568800, 1861342236135004030
9839600, 183082842898524986654160 0, 169152626591028520278300, 1462941635381868283
4880, 1179791641436990551200, 88326646952501966400, 6107693672247476400, 38779007
4428411200, 22451004309013280, 1175445251780800, 55098996177225, 2283896214600, 8
2408626300, 2535650040, 64684950, 1313400, 19900, 200, 1]
```

4. Add or double

If you start with 1 RMB and, with each move, you can either double your money or add another 1 RMB, what is the smallest number of moves you have to make to get to exactly x RMB? Here x is an integer randomly selected from 1 to 100. Write a function Least_moves to print your results. For example, Least_moves(2) should print 1, and Least_moves(5) should print 3.

In [10]:
```
import random
```

In [11]:
```
def Least_moves(t):
    n = 0
    while t != 1:
        if t % 2 != 0:
            t = t - 1
        else:
            t = t // 2
        n = n + 1

    return n
```

In [12]:
```
x = random.randint(1, 100)

print("x为： " + str(x))
print("最小操作次数为： " + str(Least_moves(x)))
```

```
x为： 31
最小操作次数为： 8
```

5. Dynamic programming

Insert + or - operation anywhere between the digits 1 2 3 4 5 6 7 8 9 in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 Write a function Find_expression, which should be able to print every possible solution that makes the expression evaluate to a random integer from 1 to 100. For example, Find_expression(50) should print lines including: [1 - 2 + 34 + 5 + 6 + 7 + 8 - 9 = 50] and [1 + 2 + 34 - 56 + 78 - 9 = 50]

In [13]:
```
import random
```

In [14]:
```
def Find_expression(target):
    digits = "123456789"

    def make_expr(index, current):
        if index == len(digits):
            if eval(current) == target:
                print(current + " = " + str(target))
            return

        make_expr(index + 1, current + "+" + digits[index])   # 加
        make_expr(index + 1, current + "-" + digits[index])   # 减
        make_expr(index + 1, current + digits[index])         # 拼接

    make_expr(1, digits[0])
```

In [15]:
```
target = random.randint(1, 100)
print(f"目标数：{target}")
Find_expression(target)
```

```
目标数：54
1+2+3-45+6+78+9 = 54
1+2+34+5+6+7+8-9 = 54
1+2+34-5-67+89 = 54
1+2-3+4+56-7-8+9 = 54
1+2-3-4-5-6+78-9 = 54
1+2-34-5-6+7+89 = 54
1+23+4-56-7+89 = 54
1+23-45+6+78-9 = 54
1-2+3+4+56-7+8-9 = 54
1-2+34+5+6-7+8+9 = 54
1-2-3-4+56+7+8-9 = 54
1-2-34+5+67+8+9 = 54
1-23-4+5+6+78-9 = 54
1-23-4+56+7+8+9 = 54
12+3+4+5+6+7+8+9 = 54
12+3-45+67+8+9 = 54
12-3+45+6-7-8+9 = 54
12-3+45-6+7+8-9 = 54
12-3-45-6+7+89 = 54
12-34-5-6+78+9 = 54
123+4-5-67+8-9 = 54
```

5.2 Count the total number of suitable solutions for any integer i from 1 to 100, assign the count to a list called Total_solutions. Plot the list Total_solutions, so which number(s) yields the maximum and minimum of Total_solutions?

In [16]:
```
import matplotlib.pyplot as plt
```

In [17]:
```
def count_solutions(target):
    digits = "123456789"
    count = 0

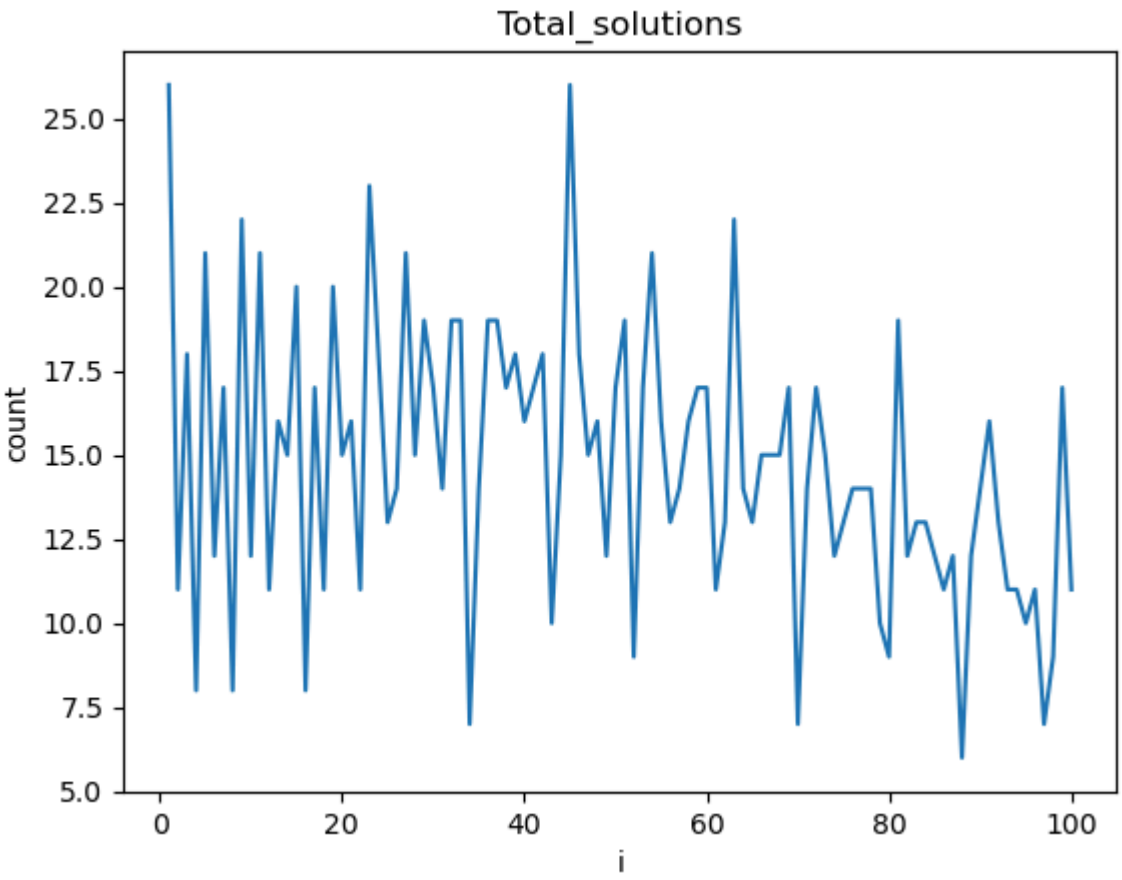
    def make_expr(index, current):
        nonlocal count
        if index == len(digits):
            if eval(current) == target:
                count = count + 1
            return
        make_expr(index + 1, current + "+" + digits[index])
        make_expr(index + 1, current + "-" + digits[index])
        make_expr(index + 1, current + digits[index])

    make_expr(1, digits[0])
    return count
```

In [18]:
```
Total_solutions = []
for i in range(1, 101):
    Total_solutions.append(count_solutions(i))
```

In [19]:
```
plt.plot(range(1, 101), Total_solutions)
plt.xlabel("i")
plt.ylabel("count")
plt.title("Total_solutions")
plt.show()

print("最大:", max(Total_solutions))
print("最小:", min(Total_solutions))
```



```
最大： 26
最小： 6
```